STRIPE PROJECT

1. Modèle de Données OLTP

D8. Exemples de requêtes SQL

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / D8 : OLTP - SQL / v25 - 2026/10/13.*

D8.1. Contexte

Exemples de requêtes sur les données transactionnelles dans les domaines suivants : analyse de revenu, détection de fraude, segmentation client.
Le modèle transactionnel physique de référence est défini dans l'annexe 1.B. Le moteur cible est *PostgreSQL*.

D8.2. Analyse de revenu

D8.2.1. Revenu et commission par marchand sur une période donnée

Indication : récupérer le montant total encaissé, la commission *Stripe* prélevée, et le revenu net du marchand, sur les 30 derniers jours, sur transactions réussies uniquement.

```sql
SELECT
    m.merchant_id, m.legal_name,
    COUNT(t.transaction_id)                 AS transactions,
    SUM(t.amount)                           AS total_amount,
    SUM(t.fee_amount)                       AS fee_amount,
    SUM(t.amount - t.fee_amount)            AS merchant_amount
FROM Transaction t
JOIN Merchant m           ON m.merchant_id = t.merchant_id
JOIN TransactionStatus ts ON ts.status_code = t.status_code
WHERE ts.status_code = 'successful'
  AND t.created_at >= now() - INTERVAL '30 days'
GROUP BY m.merchant_id, m.legal_name
ORDER BY total_amount DESC;
```

D8.2.2. Revenu par produit, ventilé entre revenu ponctuel et revenu récurrent

Indication : distinguer les transactions issues d'un abonnement des achats ponctuels (cf. suivi de performance produit exigé au cahier des charges).

```sql
SELECT
    p.product_id, p.name,
    CASE WHEN t.subscription_id IS NULL THEN 'ponctuel' ELSE 'recurrent' END AS revenu_type,
    COUNT(*)                AS revenus,
    SUM(t.amount)           AS total_amount
FROM Transaction t
JOIN Product p             ON p.product_id = t.product_id
JOIN TransactionStatus ts  ON ts.status_code = t.status_code
WHERE ts.status_code = 'successful'
GROUP BY p.product_id, p.name, revenu_type
ORDER BY p.product_id, revenu_type;
```

D8.2.3. Impact des remboursements et rétrofacturations sur le revenu net

Indication : récupérer le revenu brut, le montant remboursé, le montant en litige et le revenu net réel par marchand.

```sql
SELECT
    m.merchant_id, m.legal_name,
    SUM(t.amount)                                           AS total_amount,
```

```
    COALESCE(SUM(r.amount), 0)                                  AS refund_amount,
    COALESCE(SUM(CASE WHEN cb.status_code = 'lost' THEN t.amount ELSE 0 END), 0) AS chargeback_amount,
    SUM(t.amount) - COALESCE(SUM(r.amount), 0) - COALESCE(SUM(CASE WHEN cb.status_code = 'lost' THEN
t.amount ELSE 0 END), 0) AS actual_amount
FROM Transaction t
JOIN Merchant m       ON m.merchant_id = t.merchant_id
LEFT JOIN Refund r     ON r.transaction_id = t.transaction_id
LEFT JOIN Chargeback cb ON cb.transaction_id = t.transaction_id
WHERE t.status_code = 'successful'
GROUP BY m.merchant_id, m.legal_name
ORDER BY actual_amount DESC;
```

D8.2.4. Vérification de cohérence : `fee_amount` recalculé vs stocké

Indication : signaler les transactions où le frais fixe stocké diverge du frais recalculé à partir du plan tarifaire réellement facturé (signal un plan corrigé rétroactivement).

```
SELECT
    t.transaction_id,
    t.fee_amount                                            AS fee_stocke,
    ROUND(t.amount * pp.commission_rate + ppf.fixed_fee, 2)  AS fee_recalcule
FROM Transaction t
JOIN PricingPlan pp     ON pp.pricing_plan_id = t.pricing_plan_id
JOIN PricingPlanFee ppf ON ppf.pricing_plan_id = t.pricing_plan_id AND ppf.currency_code =
t.currency_code
WHERE ABS(t.fee_amount - (t.amount * pp.commission_rate + ppf.fixed_fee)) > 0.01;
```

D8.3. Détection de fraude

D8.3.1. Transactions à risque élevé nécessitant une revue manuelle

Indication : exploiter directement la logique portée par `FraudRiskLevel` au lieu d'utiliser un seuil arbitraire.

```
SELECT
    t.transaction_id,
    t.merchant_id,
    t.amount,
    t.currency_code,
    fs.anomaly_score,
    frl.label  AS risk_level,
    t.device_type,
    t.ip_geolocation,
    t.created_at
FROM Transaction t
JOIN FraudScore fs      ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl ON frl.risk_level_code = fs.risk_level_code
WHERE frl.requires_manual_review = true
ORDER BY fs.anomaly_score DESC;
```

D8.3.2. Taux de transactions à risque par marchand et par pays

Indication : repèrer les marchands ou zones géographiques aynt une proportion anormale de transactions à risque élevé.

```
SELECT
    m.merchant_id,
    m.legal_name,
    co.name                                          AS pays_marchand,
    COUNT(*)                                         AS nb_transactions,
    COUNT(*) FILTER (WHERE frl.blocks_transaction)   AS nb_bloquees,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE frl.blocks_transaction) / COUNT(*), 2
```

```
    )                                                   AS pct_bloquees
FROM Transaction t
JOIN Merchant m         ON m.merchant_id = t.merchant_id
JOIN Country co         ON co.country_code = m.country_code
JOIN FraudScore fs      ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl ON frl.risk_level_code = fs.risk_level_code
GROUP BY m.merchant_id, m.legal_name, co.name
HAVING COUNT(*) > 20
ORDER BY pct_bloquees DESC;
```

D8.3.3. Validation a posteriori du modèle de scoring

Indication : comparer les scores de fraude aux rétrofacturations réellement survenues (ex: un score de fraude `low` suivi d'une rétrofacturation `fraud` signale un faux négatif du modèle).

```
SELECT
    frl.label                       AS predicted_risk_level,
    COUNT(*)                        AS transactions,
    COUNT(cb.chargeback_id)         AS chargebacks,
    COUNT(cb.chargeback_id) FILTER (WHERE crc.category = 'fraud') AS frauds,
    ROUND( 100.0 * COUNT(cb.chargeback_id) FILTER (WHERE crc.category = 'fraud') / NULLIF(COUNT(*), 0),
3) AS negative_false_rate_in_percent
FROM Transaction t
JOIN FraudScore fs              ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl         ON frl.risk_level_code = fs.risk_level_code
LEFT JOIN Chargeback cb         ON cb.transaction_id = t.transaction_id
LEFT JOIN ChargebackReasonCode crc ON crc.reason_code = cb.reason_code
GROUP BY frl.label
ORDER BY negative_false_rate_in_percent DESC NULLS LAST;
```

D8.3.4. Détection de rafales sur un même moyen de paiement

Signal classique de carte compromise : plusieurs transactions à risque élevé sur le même moyen de paiement en peu de temps.

```
SELECT
    t.payment_method_id,
    COUNT(*)                    AS transactions,
    MIN(t.created_at)           AS first_transaction,
    MAX(t.created_at)           AS last_transaction,
    EXTRACT(EPOCH FROM MAX(t.created_at) - MIN(t.created_at)) / 60 AS duration_minutes
FROM Transaction t
JOIN FraudScore fs      ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl ON frl.risk_level_code = fs.risk_level_code
WHERE frl.risk_level_code IN ('medium', 'high') AND t.created_at >= now() - INTERVAL '24 hours'
GROUP BY t.payment_method_id
HAVING COUNT(*) >= 3
ORDER BY transactions DESC;
```

D8.4. Segmentation client

D8.4.1. Segmentation RFM (Recency, Frequency, Monetary value)

Indication : classer les clients d'un marchand donné en quartiles sur les trois axes RFM.

```
WITH customer_stats AS (
    SELECT
        c.customer_id,
        c.email,
        MAX(t.created_at)   AS last_transaction,
        COUNT(*)            AS transactions,
```

```
        SUM(t.amount)       AS total_amount
    FROM Customer c
    JOIN Transaction t ON t.customer_id = c.customer_id
    WHERE t.merchant_id = :merchant_id AND t.status_code = 'successful'
    GROUP BY c.customer_id
)
SELECT
    customer_id,
    email,
    transactions,
    total_amount,
    NTILE(4) OVER (ORDER BY last_transaction DESC) AS quartile_recency,
    NTILE(4) OVER (ORDER BY transactions DESC)     AS quartile_frequency,
    NTILE(4) OVER (ORDER BY total_amount DESC)     AS quartile_amount
FROM customer_stats
ORDER BY quartile_amount, quartile_frequency, quartile_recency;
```

D8.4.2. Clients abonnés à risque de désabonnement (dunning)

Indication : exploiter directement `SubscriptionStatus.triggers_dunning` et `failed_attempt_count` pour prioriser les relances par ancienneté d'échec.

```
SELECT
    c.customer_id,
    c.email,
    s.subscription_id,
    p.name                      AS produit,
    s.failed_attempt_count,
    s.current_period_end,
    ss.label                    AS statut
FROM Subscription s
JOIN SubscriptionStatus ss ON ss.status_code = s.status_code
JOIN Customer c            ON c.customer_id = s.customer_id
JOIN Product p              ON p.product_id = s.product_id
WHERE ss.triggers_dunning = true
ORDER BY s.failed_attempt_count DESC, s.current_period_end ASC;
```

D8.4.3. Segmentation par valeur et par méthode de paiement

Indication : combiner la valeur totale du client et son moyen de paiement principal.

```
SELECT
    customer_segment.segment,
    pmt.label             AS default_payment_method_type
    COUNT(*)              AS customers,
    ROUND(AVG(customer_segment.total_amount), 2) AS avg_customer_amount
FROM (
    SELECT
        c.customer_id,
        SUM(t.amount) AS total_amount
        CASE
            WHEN SUM(t.amount) >= 1000 THEN 'top'
            WHEN SUM(t.amount) >= 200  THEN 'regular'
            ELSE 'occasional'
        END AS segment
    FROM Customer c
    JOIN Transaction t ON t.customer_id = c.customer_id AND t.status_code = 'successful'
    GROUP BY c.customer_id
) customer_segment
JOIN PaymentMethod pm      ON pm.customer_id = customer_segment.customer_id AND pm.is_default = true
JOIN PaymentMethodType pmt ON pmt.type_code = pm.type_code
GROUP BY customer_segment.segment, pmt.label
ORDER BY customer_segment.segment, nb_clients DESC;
```

D8.4.4. Répartition géographique des clients par marchand

Indication : segmenter les clients par pays de résidence.

```sql
SELECT
    m.merchant_id,
    m.legal_name                  AS merchant_name,
    co.name                       AS customer_country,
    COUNT(DISTINCT c.customer_id) AS customers,
    SUM(t.amount)                 AS total_amount
FROM Customer c
JOIN Merchant m  ON m.merchant_id = c.merchant_id
LEFT JOIN Country co ON co.country_code = c.country_code
LEFT JOIN Transaction t ON t.customer_id = c.customer_id AND t.status_code = 'successful'
GROUP BY m.merchant_id, m.legal_name, co.name
ORDER BY m.merchant_id, total_amount DESC NULLS LAST;
```
